МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Брестский государственный технический университет
Кафедра ИИТ

ОТЧЕТ

по лабораторной работе №3

«Блочные шифры и классические режимы шифрования»

Выполнила:
студентка 4-го курса
группы АС-66
Демидович А. А.

Проверила:
Хацкевич А.С.

Брест 2026

Цель работы: программная реализация симметричного блочного шифра AES-128 (NIST FIPS 197) на языке высокого уровня без использования сторонних криптографических библиотек, а также реализация классических режимов шифрования ECB (Electronic Codebook) и CFB (Cipher Feedback) с демонстрацией использования исключительно функции зашифрования E_K при расшифровании данных в режиме CFB.

1. Постановка задачи

1. Реализовать базовый криптографический примитив AES-128 для строго одного 128-битного блока (функции encrypt_block, decrypt_block), включающий расширение ключа (Key Expansion) и четыре раундовых преобразования (SubBytes, ShiftRows, MixColumns, AddRoundKey).
2. Разработать алгоритм наложения (pkcs7_pad) и безопасного снятия с проверкой целостности (pkcs7_unpad) дополнения неполных блоков по стандарту PKCS#7.
3. Реализовать модуль классических режимов шифрования: обязательный базовый режим электронной кодовой книги (ECB) и дополнительный режим обратной связи по шифротексту (CFB).
4. Провести практическую демонстрацию фундаментальной особенности режима CFB: использование исключительно функции прямого преобразования E_K (encrypt_block) как при зашифровании, так и при расшифровании сообщений (без вызова D_K).
5. Выполнить верификацию шифра AES-128 по эталонным векторам стандарта NIST FIPS 197 и исследовать устойчивость к паттернам в данных (ECB vs CFB).

2. Вариант задания

Индивидуальный вариант: Вариант 3.
Базовый блочный шифр: AES-128 (размер блока 128 бит / 16 байт, размер ключа 128 бит / 16 байт, 10 раундов).
Режимы работы: ECB (Electronic Codebook) и CFB (Cipher Feedback).
Фокус тестирования и особенности реализации: реализация PKCS#7 паддинга; демонстрация использования исключительно функции E_K для расшифрования в режиме CFB; сравнение стойкости к паттернам в данных (ECB vs CFB).

Таблица 1 — Исходные параметры и характеристики шифра (Вариант 3)

Параметр	Значение	Описание / Стандарт
Шифр	AES-128	Симметричная SP-сеть (NIST FIPS 197)

Размер блока (B)	128 бит (16 байт)	Матрица состояния State 4×4 байта
Длина ключа (K)	128 бит (16 байт)	10 раундов шифрования (11 подключей)
Режимы шифрования	ECB, CFB	Электронная кодовая книга и обратная связь по шифротексту
Особенность CFB	Только E_K	При дешифровании функция D_K не применяется

3. Архитектура программного решения и теоретические основы режима CFB

Режим CFB (Cipher Feedback) преобразует блочный шифр в самосинхронизирующийся потоковый шифр. Вектор инициализации IV шифруется с помощью функции прямого преобразования E_K для выработки первой порции гаммы, которая складывается по модулю 2 с блоком открытого текста. Полученный блок шифротекста C_i подается на вход функции шифрования E_K для генерации гаммы следующего блока:

• Зашифрование (C_0 = IV):
$C_i = P_i \oplus E_K(C_{i-1}),\ i = 1, ..., N$

• Расшифрование (C_0 = IV):
$P_i = C_i \oplus E_K(C_{i-1}),\ i = 1, ..., N$

Математическое следствие: при расшифровании для вычисления гаммы требуется зашифровать предыдущий блок шифротекста C_{i-1}, поэтому функция обратного раунда D_K (decrypt_block) в режиме CFB не используется вовсе. Это упрощает аппаратную и программную реализацию, так как нет необходимости хранить обратные таблицы InvS-box и матрицы InvMixColumns для потокового дешифрования.

4. Листинг ключевых функций

4.1. Реализация режима CFB с применением исключительно функции E_K (cipher_modes.py)

```
def encrypt_cfb(plaintext: bytes, key: bytes, iv: bytes) -> bytes:
    """Зашифрование в режиме CFB: Ci = Pi ^ E_k(Ci-1), C0 = IV."""
    if len(iv) != BLOCK_SIZE:
        raise ValueError("Размер IV должен быть ровно 16 байт.")
```

```
    padded = pkcs7_pad(plaintext, BLOCK_SIZE)
    ciphertext = bytearray()
    prev_block = iv

    for i in range(0, len(padded), BLOCK_SIZE):
        block = padded[i : i + BLOCK_SIZE]
        # Генерация гаммы с помощью прямой функции E_k
        keystream = encrypt_block(prev_block, key)
        c_block = xor_bytes(block, keystream)
        ciphertext.extend(c_block)
        prev_block = c_block

    return bytes(ciphertext)

def decrypt_cfb(ciphertext: bytes, key: bytes, iv: bytes) -> bytes:
    """Расшифрование в режиме CFB: Pi = Ci ^ E_k(Ci-1), C0 = IV.
    ДЕМОНСТРАЦИЯ: используется ТОЛЬКО encrypt_block (E_k), decrypt_block НЕ вызывается!"""
    if len(ciphertext) % BLOCK_SIZE != 0:
        raise ValueError("Длина шифротекста в CFB должна быть кратна размеру блока.")
    if len(iv) != BLOCK_SIZE:
        raise ValueError("Размер IV должен быть ровно 16 байт.")

    padded_plaintext = bytearray()
    prev_block = iv

    for i in range(0, len(ciphertext), BLOCK_SIZE):
        c_block = ciphertext[i : i + BLOCK_SIZE]
        # Гамма восстанавливается повторным вызовом encrypt_block (E_k) над предыдущим
шифроблоком
        keystream = encrypt_block(prev_block, key)
        p_block = xor_bytes(c_block, keystream)
        padded_plaintext.extend(p_block)
        prev_block = c_block

    return pkcs7_unpad(bytes(padded_plaintext), BLOCK_SIZE)
```

4.2. Реализация базового режима ECB (cipher_modes.py)

```
def encrypt_ecb(plaintext: bytes, key: bytes, block_size: int = 16) -> bytes:
    padded = pkcs7_pad(plaintext, block_size)
    ciphertext = bytearray()
    for i in range(0, len(padded), block_size):
        block = padded[i : i + block_size]
        ciphertext.extend(encrypt_block(block, key))
    return bytes(ciphertext)

def decrypt_ecb(ciphertext: bytes, key: bytes, block_size: int = 16) -> bytes:
    if len(ciphertext) % block_size != 0:
        raise ValueError("Длина шифротекста в ECB должна быть кратна размеру блока.")
    padded_plaintext = bytearray()
    for i in range(0, len(ciphertext), block_size):
        block = ciphertext[i : i + block_size]
        padded_plaintext.extend(decrypt_block(block, key))
    return pkcs7_unpad(bytes(padded_plaintext), block_size)
```

4.3. Функции наложения и валидации паддинга PKCS#7 (cipher_modes.py)

```
def pkcs7_pad(data: bytes, block_size: int = 16) -> bytes:
    remainder = len(data) % block_size
    pad_len = block_size - remainder
    return data + bytes([pad_len] * pad_len)

def pkcs7_unpad(data: bytes, block_size: int = 16) -> bytes:
    if not data or len(data) % block_size != 0:
        raise ValueError("Некорректный размер блока данных для удаления паддинга.")
    pad_len = data[-1]
    if pad_len < 1 or pad_len > block_size:
        raise ValueError(f"Недопустимое значение байта паддинга: {pad_len}")
    expected_pad = bytes([pad_len] * pad_len)
    if data[-pad_len:] != expected_pad:
        raise ValueError("Нарушена структура паддинга PKCS#7 (Padding Error).")
    return data[:-pad_len]
```

5. Результаты выполнения и тестирования работы

5.1. Верификация базового алгоритма AES-128 по вектору NIST FIPS 197

Для верификации базового шифра использован официальный тестовый вектор NIST FIPS 197 (Appendix B):

- Исходный ключ (Key): 2b7e151628aed2a6abf7158809cf4f3c
- Открытый текст (Plaintext): 3243f6a8885a308d313198a2e0370734
- Ожидаемый шифротекст (Expected CT): 3925841d02dc09fbdc118597196a0b32
- Полученный шифротекст (Received CT): 3925841d02dc09fbdc118597196a0b32

Результат: зафиксировано полное побайтовое совпадение с эталоном. Базовый шифр AES-128 полностью корректен.

5.2. Тестирование механизма дополнения блоков PKCS#7

Таблица 2 — Результаты тестирования паддинга PKCS#7 для различных длин

Сценарий	Длина текста (L)	Длина с Pad	Добавлено (k)	Байты дополнения (HEX)
Меньше 16 байт ('Hello World')	11 байт	16 байт	5 байт	05 05 05 05 05
Ровно 16 байт ('0123456789ABCDEF')	16 байт	32 байта	16 байт	10 10 ... 10 (16 раз)
Больше 16 байт ('Cryptography course...')	25 байт	32 байта	7 байт	07 07 07 07 07 07 07

5.3. Сравнительный анализ стойкости к сохранению паттернов: ECB vs CFB

Для тестирования сформирован открытый текст из четырех идентичных 16-байтовых блоков: b'SECRET_BLOCK_16B' * 4 (64 байта). В таблице 3 приведено поблочное сравнение шифротекстов в режимах ECB и CFB.

Таблица 3 — Поблочное сравнение шифротекстов режимов ECB и CFB

Блок	Открытый текст (HEX)	Шифротекст ECB (HEX)	Шифротекст CFB (HEX)
0	534543524554... (Блок 1)	134a9f680229cb302b1950ec00be6e24	7e813df9a45b736c05d761bc93ef6420
1	534543524554... (Блок 2)	134a9f680229cb302b1950ec00be6e24	b40d65b1ae431b90df7f7396c781 10b5
2	534543524554... (Блок 3)	134a9f680229cb302b1950ec00be6e24	92c4f738a16db8a20d6f43e110b9d7aa
3	534543524554... (Блок 4)	134a9f680229cb302b1950ec00be6e24	1fa930e84b2386a5ecf4886618705c74
4 (Pad)	10 10 10 ... 10 (16 байт)	0fb05060af6d053fa0e78d5d56e0b761	5a8e0cb26e25785f7560d0e2e92c57a9

6. Анализ полученных результатов

Анализ проведенных экспериментов подтверждает ключевые теоретические особенности Варианта 3:

1. Исключительное использование E_K при расшифровании в CFB: в кодовой реализации функции decrypt_cfb полностью отсутствует вызов decrypt_block (D_K). Для восстановления исходного текста блок шифротекста складывается по модулю 2 с гаммой, порожденной зашифрованием предыдущего блока шифротекста: $P_i = C_i \oplus E_K(C_{i-1})$. Эксперимент подтвердил 100% обратимость операции расшифрования без использования обратного преобразования блочного шифра.
2. Сравнение стойкости к паттернам: в режиме ECB одинаковые блоки открытого текста дают побайтово идентичный шифротекст (134a9f680229cb302b1950ec00be6e24), раскрывая структуру и закономерности данных. В режиме CFB благодаря обратной связи и начальному вектору IV все блоки шифротекста (7e813d..., b40d65..., 92c4f7..., 1fa930...) абсолютно уникальны и псевдослучайны.
3. Надежность PKCS#7: механизм дополнения гарантирует корректное и однозначное восстановление исходной границы полезной нагрузки.

Вывод

В ходе выполнения лабораторной работы были изучены принципы функционирования симметричного блочного шифра AES-128 и режимов шифрования ECB и CFB согласно заданию Варианта 3. Самостоятельно на языке Python без сторонних криптографических библиотек реализован алгоритм AES-128, верифицированный официальным тестом NIST FIPS 197. Реализованы модуль дополнения PKCS#7 и режимы ECB и CFB. Экспериментально подтверждена ключевая теоретическая особенность режима CFB: возможность дешифрования произвольных данных с использованием исключительно функции зашифрования блока E_K. Продемонстрирована уязвимость режима ECB к сохранению паттернов и высокая стойкость режима CFB к статистическому анализу.